Progress Report: Secure Multi-Tenant Identity, Authorization and Enterprise Access Platform

Krishna Teja (CS23BTECH11028) Jaya Prakash (CS23BTECH11015)

October 2026

A. Proposed Approach

Our project tackles the challenge of building a secure, multi-tenant learning platform where users can have different roles across various organizations. To address this, we decided to use a modular monolith architecture. This approach is better than managing multiple microservices because it reduces the complexity for our small team while still keeping the code organized in logical modules like authentication, user management, and resources.

For identity management, we chose to integrate Keycloak (version 26.8.0). Building a custom identity provider is risky and takes too much time, which our mentors pointed out as a major risk. Keycloak handles OIDC login, token issuance, and session lifecycles out of the box, allowing us to focus on the actual business rules.

For authorization, we evaluated Keycloak's built-in policies but found them awkward for our tenant-specific and relationship-based rules (like "an instructor can only grade courses they teach"). Therefore, we integrated Casbin as our declarative authorization engine. This allows us to define clear, testable role-based policies while keeping the tenant and relationship facts in our application database (PostgreSQL). We don't merge accounts by email; instead, we strictly use the issuer and subject to uniquely identify users.

B. System Design

The system consists of several interacting components deployed via Docker Compose to ensure a reproducible environment:

- **Clients:** We have lightweight web and mobile-style browser clients. They use Authorization Code Flow with PKCE to authenticate securely. The client never handles passwords directly; everything goes through Keycloak.
- **Identity Provider (Keycloak):** Manages user credentials, issues JWT access tokens, and handles enterprise federation via an OIDC broker. It runs on its own private PostgreSQL database.
- **Backend API (Fastify):** A Node.js server that validates incoming tokens using Keycloak's JWKS. It uses custom middleware to map the token's subject to our internal user ID and verifies the requested tenant context.
- **Database (PostgreSQL):** Stores application state including users, organizations, memberships, learning resources, submissions, consent, and queryable audit logs. It enforces data integrity using unique constraints and composite foreign keys to prevent cross-tenant data leaks.

Workflow Details: A user logs in via the client and gets an access token from Keycloak. The client sends this token and an `X-Organization-ID` header to the Fastify API. The backend verifies the token signature and checks if the token is an access token (rejecting ID tokens). Then, it checks if the user has an active membership in that organization. After that, Casbin evaluates if the user's role permits the requested action, with explicit deny rules taking precedence. Finally, the API performs the action and logs an audit event in the same database transaction.

C. Technical Feasibility

Tools and Frameworks:

- Node.js (v24.21.0), Fastify (v5.12.5) for the backend API.
- PostgreSQL (v16.15) with Kysely for type-safe database interactions and migrations.
- Keycloak (v26.8.0) for identity management and session handling.
- Casbin (v5.51.1) for declarative RBAC authorization.
- Docker and Docker Compose for reproducible local infrastructure.
- Vitest (v5.0.3) and Playwright for automated security and browser testing.

Feasibility and Challenges: The design is technically feasible as it relies on mature, well-supported open-source tools. One major challenge was handling cross-tenant data leaks. We mitigated this by ensuring every database query is strictly filtered by both the resource ID and the selected organization ID.

Another challenge was ensuring that token revocation works properly. We found that a password reset in Keycloak does not automatically revoke an existing access session. So incident response must explicitly trigger a logout-all to be safe. We also had to make sure that local database failures (like an audit log failing to insert) roll back the entire transaction, which we achieved using PostgreSQL transactions.

D. Implementation Plan

Completed Work:

- Infrastructure setup with Docker Compose, separating the application database from Keycloak's database.
- Database migrations with composite constraints, organization quotas, and initial synthetic seed data (e.g., Acme and Globex organizations).
- Fastify backend with strict JWT token validation, tenant context middleware, and transactional audit logging.
- Casbin integration for declarative RBAC policies, handling 48 different role/resource/action combinations.
- API endpoints for learning resources, submissions (including grading), member management, and consent tracking.
- Automated security test suite (159 tests passing, covering authorization, tenant isolation, and live Keycloak session lifecycle).
- Real-browser PKCE login verification using Playwright.

Ongoing and Pending Tasks:

- *Ongoing:* Refining the documentation, including the final traceability matrix and demo guides.
- *Pending:* Implementation of optional extensions such as application machine identities (service principals), user-delegated agents, and full SCIM provisioning.
- *Pending:* Further hardening for production deployment, such as secret management and TLS configuration (currently everything runs locally for the academic prototype).

Expected Outcomes: The expected outcome is a fully functional, secure prototype demonstrating proper tenant isolation, robust authorization, and clear audit trails, ready for final evaluation and demonstration to our mentors. We have a working baseline that proves our approach is sound.